DEVSECOPS

1. Setup DevSecOps pipeline using freestyle project.

source code: rajkumari-hub/spring-petclinic.git

DevSecOps Pipeline Setup Using Jenkins Freestyle Project

We will create a hands-on DevSecOps project using your source code repository:

Source code: spring-petclinic GitHub repository

The goal is to configure a Jenkins Freestyle Project that downloads the source code, builds the application, performs code-quality and security checks, and stores the build artifact.

1. DevSecOps pipeline architecture

1. Source Code — GitHub

spring-petclinic repository

2. Jenkins Freestyle Project

Checkout code and trigger build

3. Build

Maven compile and package

4. Security

OWASP Dependency-Check

5. Code Quality

SonarQube analysis

6. Artifact

Jenkins archive / Nexus

We will configure these steps through Jenkins GUI, without writing a Jenkinsfile. The first objective is to get the build working; then we will integrate the security and quality tools.

2. Prerequisites

You need the following components:

Component	Purpose
Jenkins	Freestyle job and build automation
Git	Clone the GitHub repository
JDK	Run the Java application build
Maven	Compile, test and package the application
SonarQube	Static code-quality analysis
OWASP Dependency-Check	Scan third-party dependencies for known vulnerabilities
Nexus Repository (optional)	Store build artifacts

You previously shared Jenkins, SonarQube and Nexus server URLs. You can reuse those servers if they are still running and accessible:

- Jenkins: http://3.82.59.232:8080
- SonarQube: http://44.203.82.139:9000
- Nexus: http://13.218.184.207:8081

Important: These are the URLs you previously provided, not a live availability check. Before starting, verify that each service is running and that its security group permits access only from the appropriate trusted sources.

3. Step 1 — Verify Jenkins and install plugins

Open your Jenkins URL:

http://23.20.228.199:8080

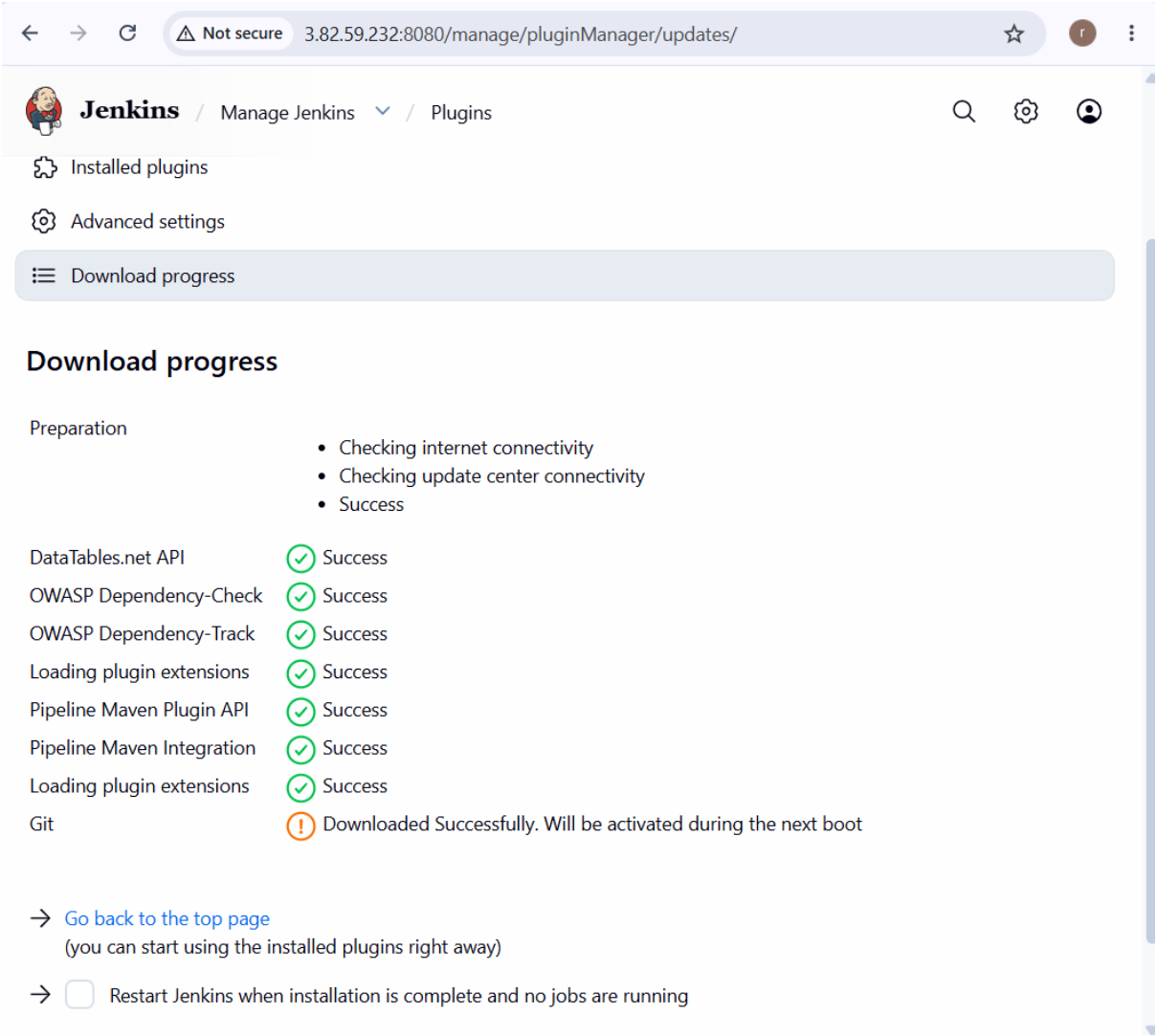
1. Log in to Jenkins.
2. Select Manage Jenkins.
3. Open Plugins or Manage Plugins, depending on your Jenkins version.
4. Open the Available plugins section.

5. Search for and install these plugins:

Plugin	Purpose
Git	Checkout source code from GitHub
Maven Integration	Provides Maven build steps
SonarQube Scanner	Connects Jenkins with SonarQube
OWASP Dependency-Check	Dependency vulnerability scanning

Install the plugins and restart Jenkins if requested. Check the Installed plugins page to confirm they are available.

The SonarQube plugin supports Freestyle jobs, and Dependency-Check provides a build step and report publishing functionality



4. Step 2 — Verify Java, Maven and Git

Connect to the server or VM where Jenkins executes builds. If Jenkins runs on an EC2 Linux instance, connect using SSH.

Run:

java -version

mvn -version

git --version

You should see the installed versions of Java, Maven and Git.

If a command returns command not found, install the missing software on the Jenkins build machine.

For a RHEL/Amazon Linux machine, you can use the following commands as appropriate for your OS:

sudo dnf install -y git java-21-amazon-corretto-devel maven

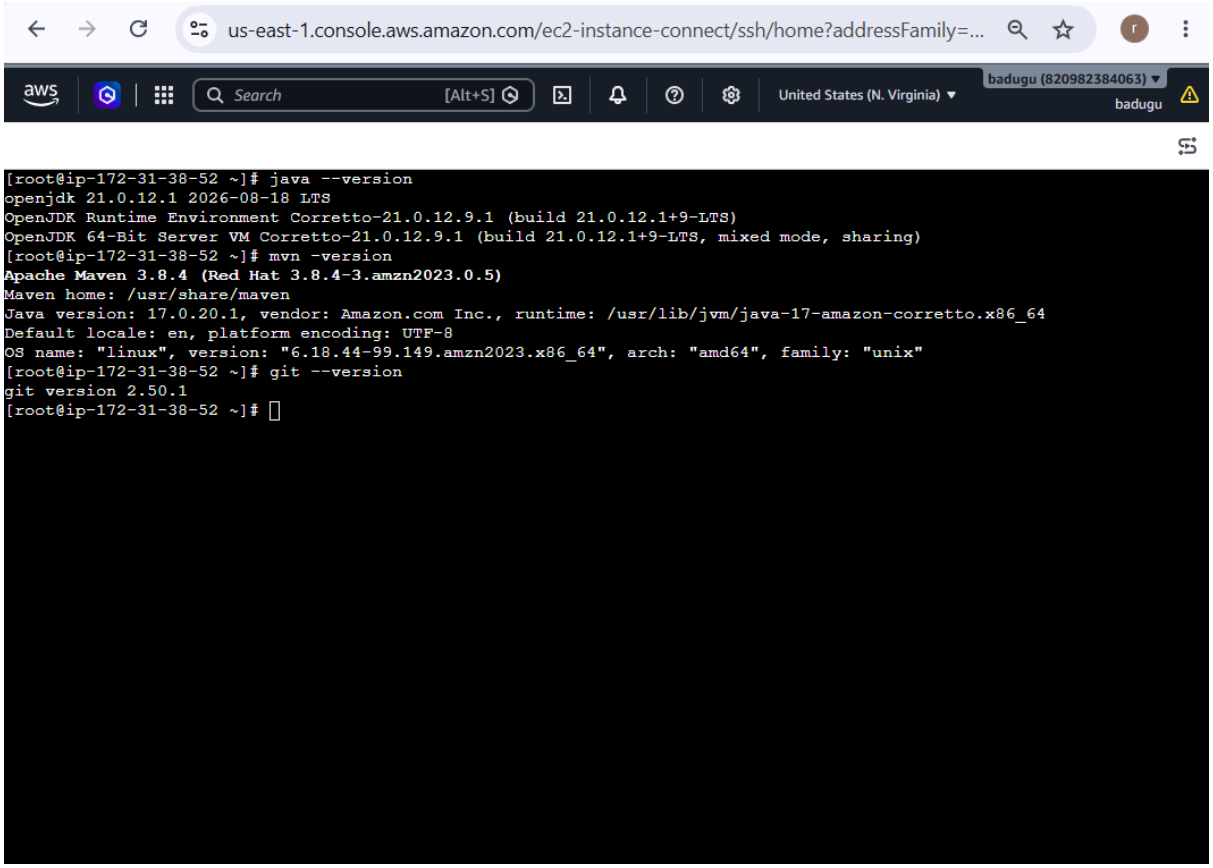
Then verify again:

java -version

mvn -version

git --version

Important: Confirm the Java version required by the repository before selecting a JDK. If the project specifies another supported Java version, use that version rather than blindly changing Java on an existing server.



5. Step 3 — Create the Jenkins Freestyle project

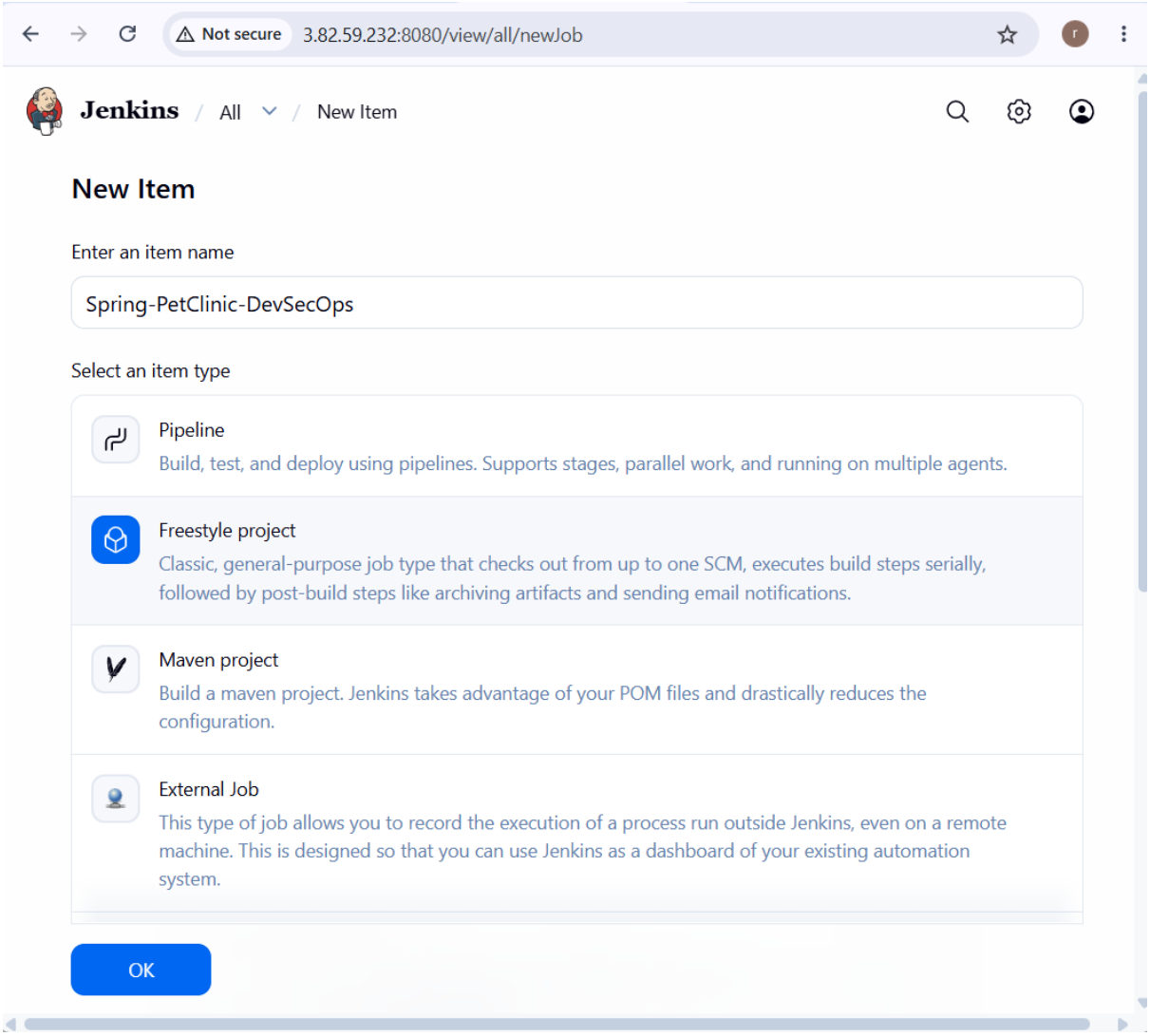
1. Open Jenkins Dashboard

Click New Item in the left-side menu.

2. Enter the job name

Enter Spring-PetClinic-DevSecOps, select Freestyle project, and click OK.

You are now on the job configuration page.



6. Step 4 — Configure source code management

Scroll to Source Code Management.

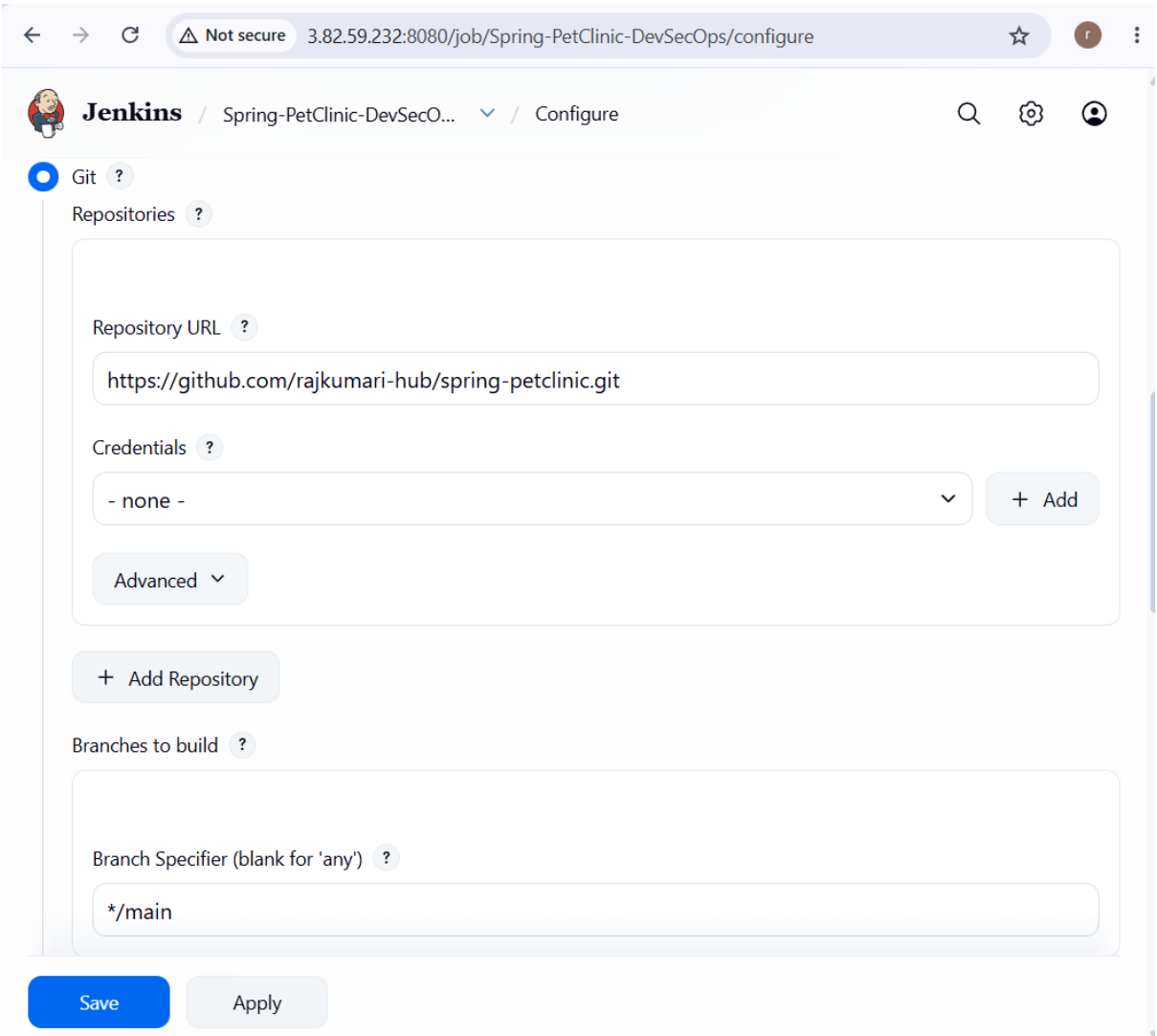
Select Git and enter the following details:

Field	Value
Repository URL	https://github.com/rajkumari-hub/spring-petclinic.git
Credentials	None, if the repository is publicly accessible
Branch Specifier	*/main

If the repository uses a different default branch, select the branch that actually exists.

Under Build Triggers, select GitHub hook trigger for GITScm polling only if you plan to configure a GitHub webhook. Otherwise, you can initially leave the trigger unchecked and build manually.

Click Save only after finishing the remaining configuration, or continue down the page.



7. Step 5 — Configure the build using Maven

Scroll to the Build Steps section.

Click Add build step.

If you see Invoke top-level Maven targets, you can use it. Otherwise, select Execute shell on Linux or Execute Windows batch command on Windows.

For a Linux Jenkins agent, use Execute shell and enter:

```
set -eu
```

```
echo "===== Java Version ====="
java -version

echo "===== Git Version ====="
git --version

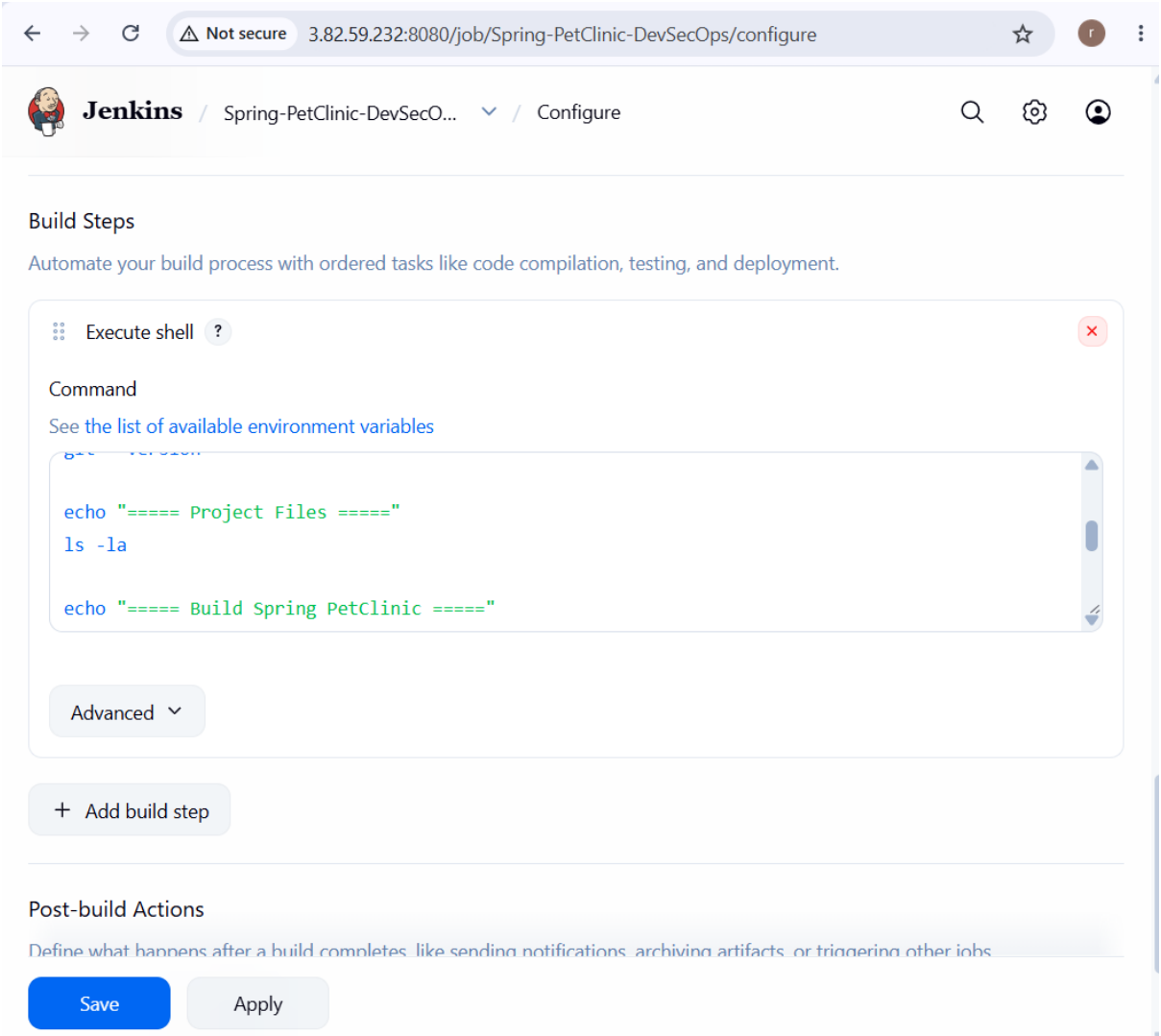
echo "===== Project Files ====="
ls -la

echo "===== Build Spring PetClinic ====="
if [ -f ./mvnw ]; then
    chmod +x ./mvnw
    ./mvnw -B clean verify
else
    mvn -B clean verify
fi

echo "===== Generated Artifacts ====="
find . -path '*/target/*.jar' -type f -print
```

This command checks for the Maven Wrapper (mvnw) and uses it if available. The verify lifecycle compiles the project, runs the applicable tests and performs verification.

If the build fails because of an incompatible JDK, inspect the repository's Java and Maven configuration before changing the command.



8. Step 6 — Add the OWASP Dependency-Check security scan

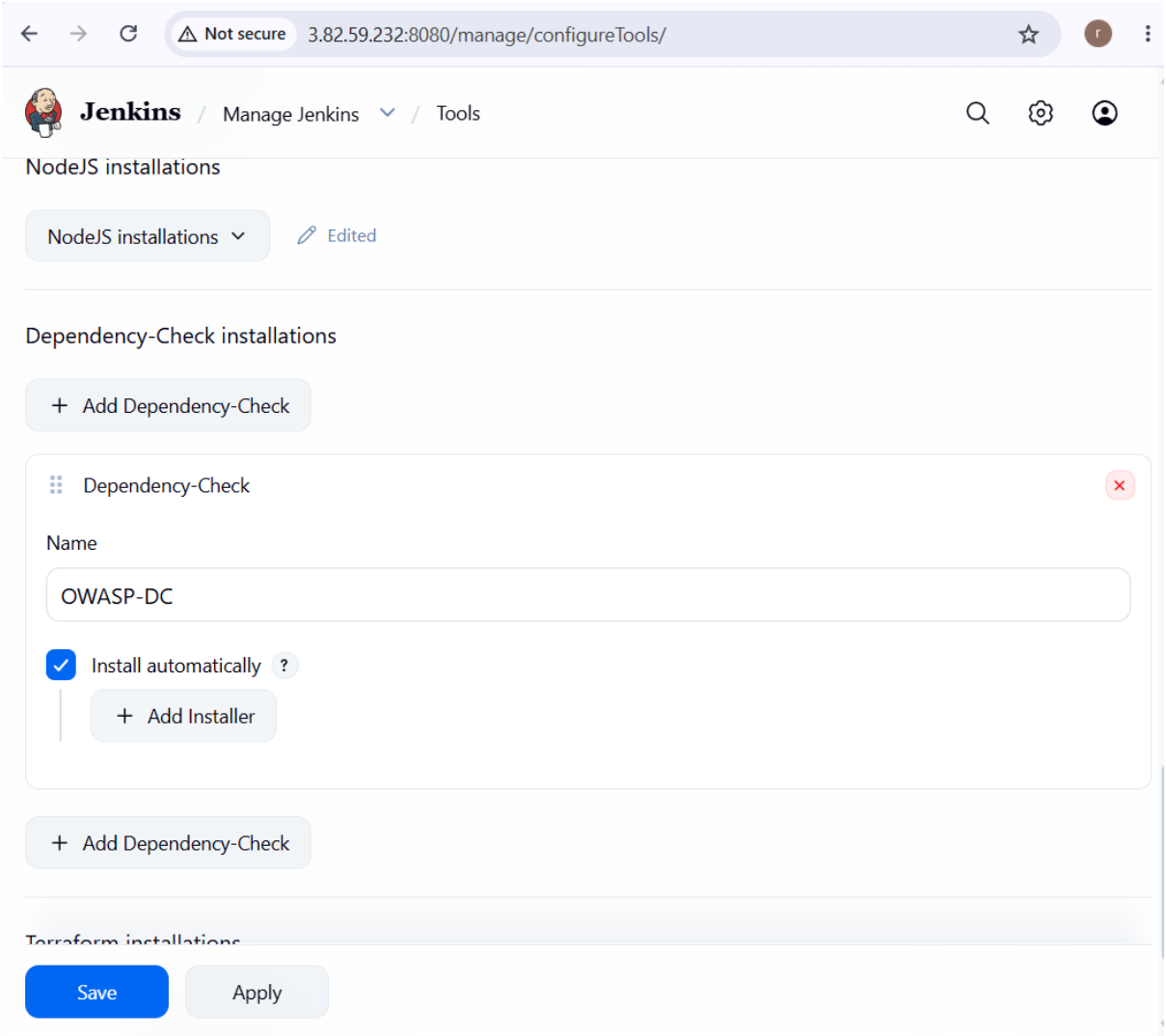
Dependency-Check identifies known vulnerabilities in third-party components.

OWASP Foundation

First, install and configure the scanner:

1. Go to Manage Jenkins → Tools.
2. Find the Dependency-Check installations section.
3. Click Add Dependency-Check.
4. Enter a name such as OWASP-DC.
5. Enable automatic installation if that option is available.
6. Save the configuration.

The exact options depend on the installed plugin version.



Next, open your Freestyle job and select Configure.

Under Build Steps, add the Invoke Dependency-Check step and select OWASP-DC.

If the plugin is not available or you prefer the Maven-based method, add another Execute shell step after your Maven build:

```
set -eu


if [ -f ./mvnw ]; then
    ./mvnw -B org.owasp:dependency-check-maven:check
else
    mvn -B org.owasp:dependency-check-maven:check
fi
```

This alternative invokes the Dependency-Check Maven plugin. It may require a first-run database download, network access and an appropriate NVD API key, depending on your configuration and plugin version.

Do not add both scan methods unnecessarily. Use one method and ensure the generated report is retained.

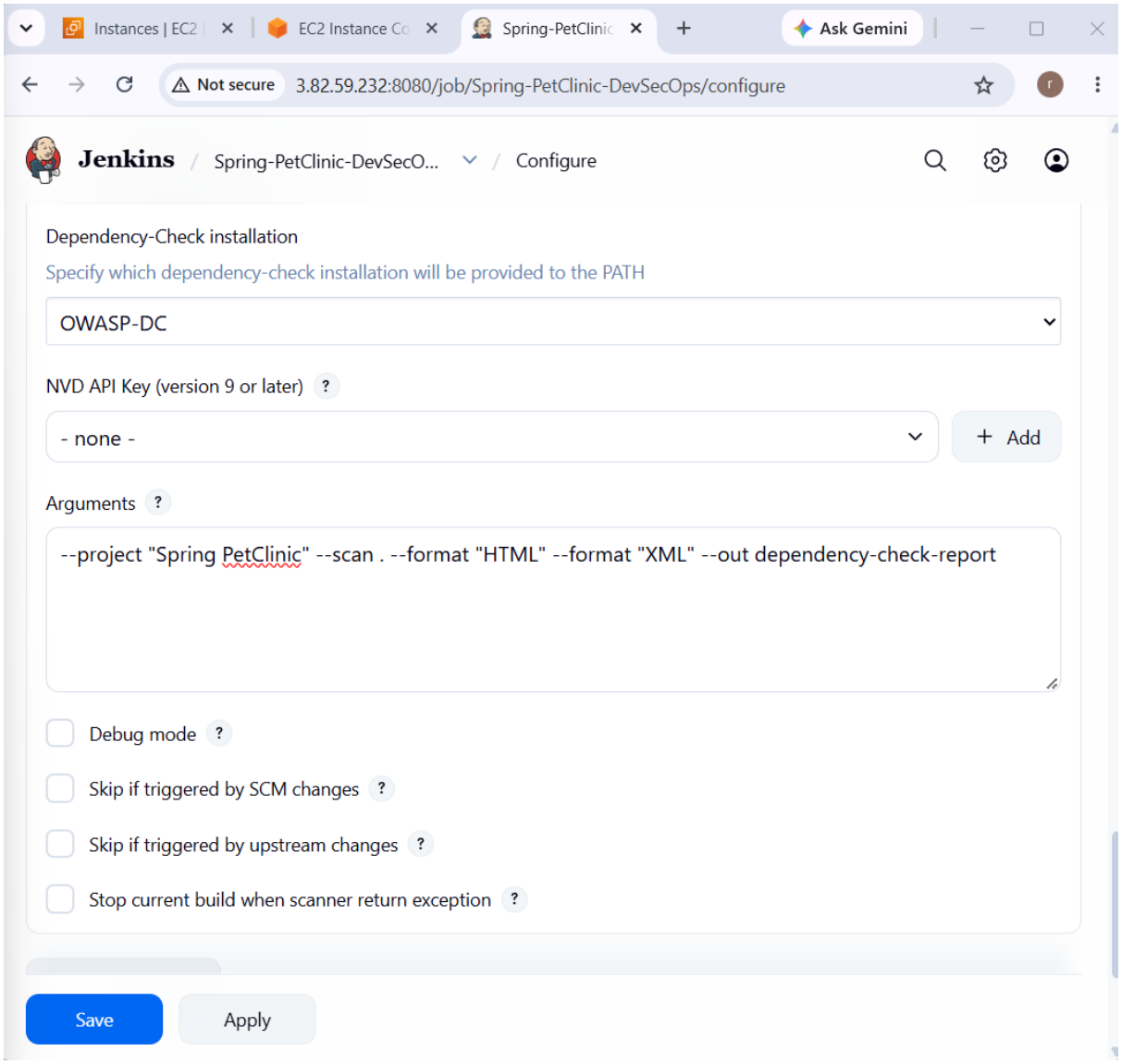
Publish the security report

In the Freestyle job's post-build actions, add the Dependency-Check publisher if available. Configure the report pattern to match the XML report produced by your chosen scanner.

For example, if the report is generated at the workspace root:

dependency-check-report.xml

The actual report path may differ, so verify it in the console output before configuring the publisher.



9. Step 7 — Configure SonarQube code-quality analysis

Your SonarQube server URL is:

http://44.203.82.139:9000

A. Generate a SonarQube token

1. Log in to SonarQube.
2. Open your account's security settings.
3. Generate a token for Jenkins analysis.
4. Copy the token and store it securely.

Never paste the token directly into a shell command or commit it to GitHub.

B. Add the token to Jenkins credentials

1. Open Manage Jenkins → Credentials.
2. Select the appropriate global or system credential store.
3. Click Add Credentials.
4. Select Secret text.
5. Paste the SonarQube token into the Secret field.
6. Set an ID such as sonarqube-token.
7. Save.

C. Configure the SonarQube server

1. Go to Manage Jenkins → System.
2. Find SonarQube servers.
3. Enable environment-variable injection if the option is shown.
4. Add a server configuration:
    - Name: SonarQube
    - Server URL: http://44.203.82.139:9000
    - Server authentication token: select your stored token.
5. Save.

Configure the SonarScanner tool under Manage Jenkins → Tools if your chosen analysis method requires it.

D. Add the analysis step to the Freestyle job

SonarQube supports Maven-based analysis in Freestyle jobs.

Sonar Documentation

In the job configuration:

1. Go to Build Environment.
2. Enable Prepare SonarScanner environment and select SonarQube, if available.
3. Add a Maven build step or an Execute shell step after the build.

For a project using Maven and a compatible SonarQube Maven scanner, an example command is:

```
./mvnw -B sonar:sonar \
  -Dsonar.host.url=http://32.192.23.255:9000 \

  -Dsonar.projectKey=spring-petclinic \
  -Dsonar.projectName=Spring-PetClinic
```

Use mvn instead of ./mvnw if the repository does not include the wrapper.

Important: The command above assumes that the token is securely injected by your Jenkins/SonarQube configuration and that the Maven scanner version is compatible with the server. Do not execute it until authentication is configured. Also, a successful analysis submission is not the same as a passing Quality Gate; configure a separate quality-gate check if your setup supports it.

10. Step 8 — Archive the build artifact

After a successful build, Jenkins should retain the generated JAR file.

In your job configuration:

1. Find Post-build Actions.
2. Click Add post-build action.
3. Select Archive the artifacts.
4. Enter this pattern:

```
**/target/*.jar
```

5. Enable Allow empty archive only if you intentionally want to allow builds without JAR files; normally, leave it disabled.
6. Save the job.

Jenkins will archive matching JAR files so you can retrieve them from the build page.

If you also want to upload the artifact to your Nexus server, that can be added after the basic build and scans work. Your Nexus URL is http://13.218.184.207:8081; use a dedicated Nexus credential stored in Jenkins, rather than putting a username or password in the job command.

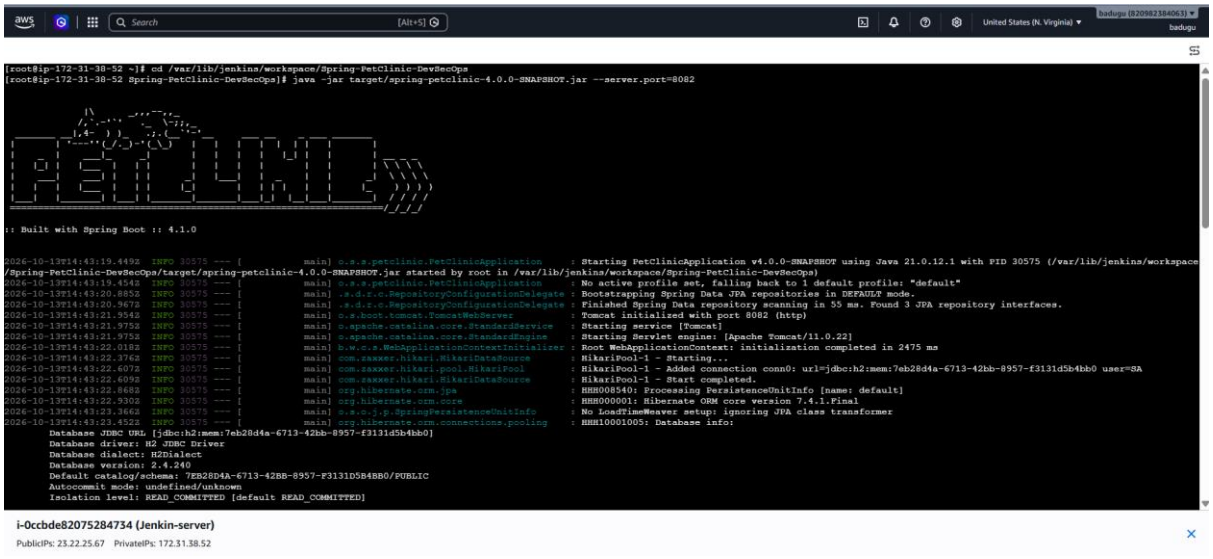
11. Step 9 — Save and run the job

1. Click Save.
2. On the job page, click Build Now.
3. Open the new build number.
4. Click Console Output.
5. Check the log for each step.

A successful run should show evidence of:

- Git checkout completed.
- Maven build and tests completed.
- Dependency-Check scan completed and report generated.
- SonarQube analysis submitted successfully.
- Artifact archiving completed.

The job may fail if any configured security or quality tool fails. That is expected when the tool is enforcing a build policy; inspect the specific step rather than disabling the scan immediately.



Not secure 32.192.23.255:9000/projects

Embedded database should be used for evaluation purposes only. It doesn't support scaling, upgrading to a new SonarQube Server version, or migration to another database engine. Learn more

Keep your instance current and get the latest SonarQube Community Build , available now.

The way in which security, reliability, and maintainability counts and ratings are calculated has changed. Learn more in SonarQube documentation

SonarQube community

Projects Issues Rules Quality profiles Quality gates Administration More

My Favorites All

Filters

Quality Gate

Passed 2

Failed 0

Security

≥ 0 info issues 2

≥ 1 low issue 0

≥ 1 medium issue 0

≥ 1 high issue 0

≥ 1 blocker issue 0

Reliability

amazon Public

Last analysis: 4 days ago

Passed

The main branch of this project is empty.

Spring-PetClinic Public

Last analysis: 44 minutes ago · 10k Lines of Code · CSS, Java, ...

Passed

A 0 Security

C 49 Reliability

A 57 Maintainability

A — Hotspots Reviewed

92.3% Coverage

0.0% Duplications

2 of 2 shown

SonarQube™ technology is powered by SonarSource Sàrl Community Build · v26.6.0.123539 · MQR MODE LGPL v3 Community Documentation Plugins Web API

Not secure 23.22.25.67:8082

spring HOME FIND OWNERS VETERINARIANS ERROR

Welcome



spring by VMware Tanzu

Not secure 23.22.25.67:8082/owners?lastName=

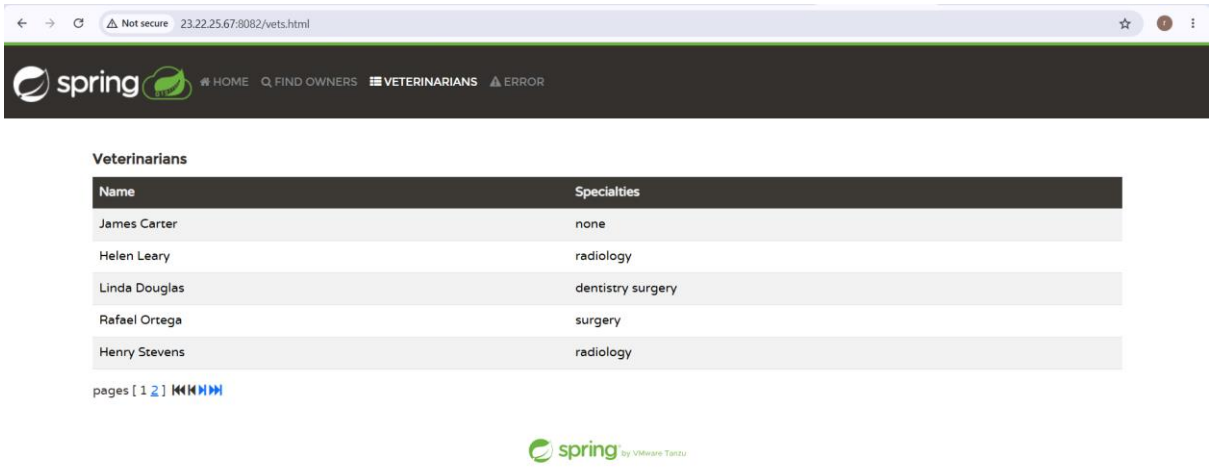
spring HOME FIND OWNERS VETERINARIANS ERROR

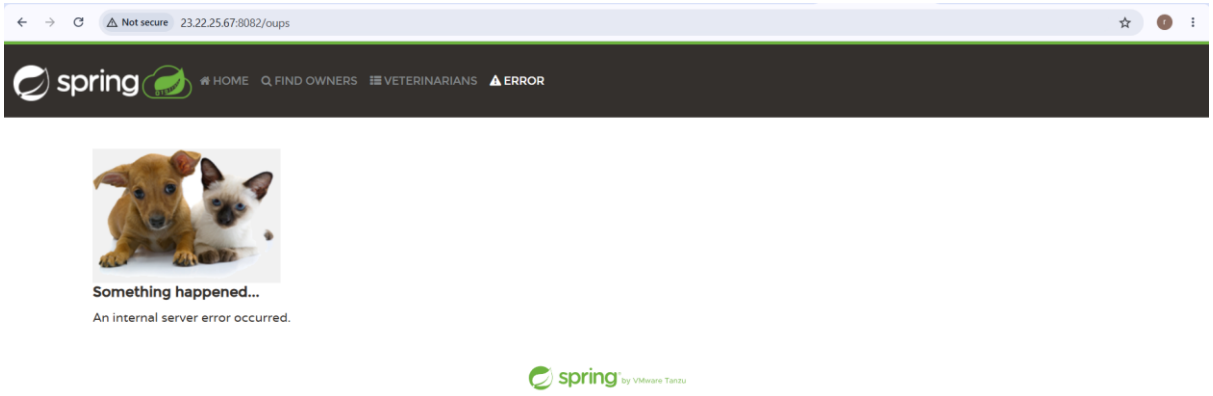
Owners

Name	Address	City	Telephone	Pets
George Franklin	110 W. Liberty St.	Madison	6085551023	Leo
Betty Davis	638 Cardinal Ave.	Sun Prairie	6085551749	Basil
Eduardo Rodriquez	2693 Commerce St.	McFarland	6085558763	Jewel, Rosy
Harold Davis	563 Friendly St.	Windsor	6085553198	Iggy
Peter McTavish	2387 S. Fair Way	Madison	6085552765	George

pages [1 2]

spring by VMware Tanzu





2. Setup DevSecOps pipeline using Declarative pipeline.

source code: [rajkumari-hub/spring-petclinic.git]

We will use a Jenkins Declarative Pipeline with a Jenkinsfile to automate the Spring PetClinic build, testing, security scanning, code-quality analysis, and artifact archiving. Unlike a Freestyle project, the stages and steps are defined as code.

1. Pipeline architecture

1Checkout

Download code from GitHub

2Build

Compile using Maven

3 Test

Run unit and integration tests

4 Security Scan

OWASP Dependency-Check

5 Code Quality

SonarQube analysis

6 Quality Gate

Check quality status

7 Publish Artifact

Archive JAR in Jenkins; optionally upload to Nexus

We will first get the pipeline working through the build and test stages, then integrate security tools. This makes errors easier to troubleshoot.

2. Prerequisites

You previously provided these service URLs. Reuse them if the services are still running and reachable.

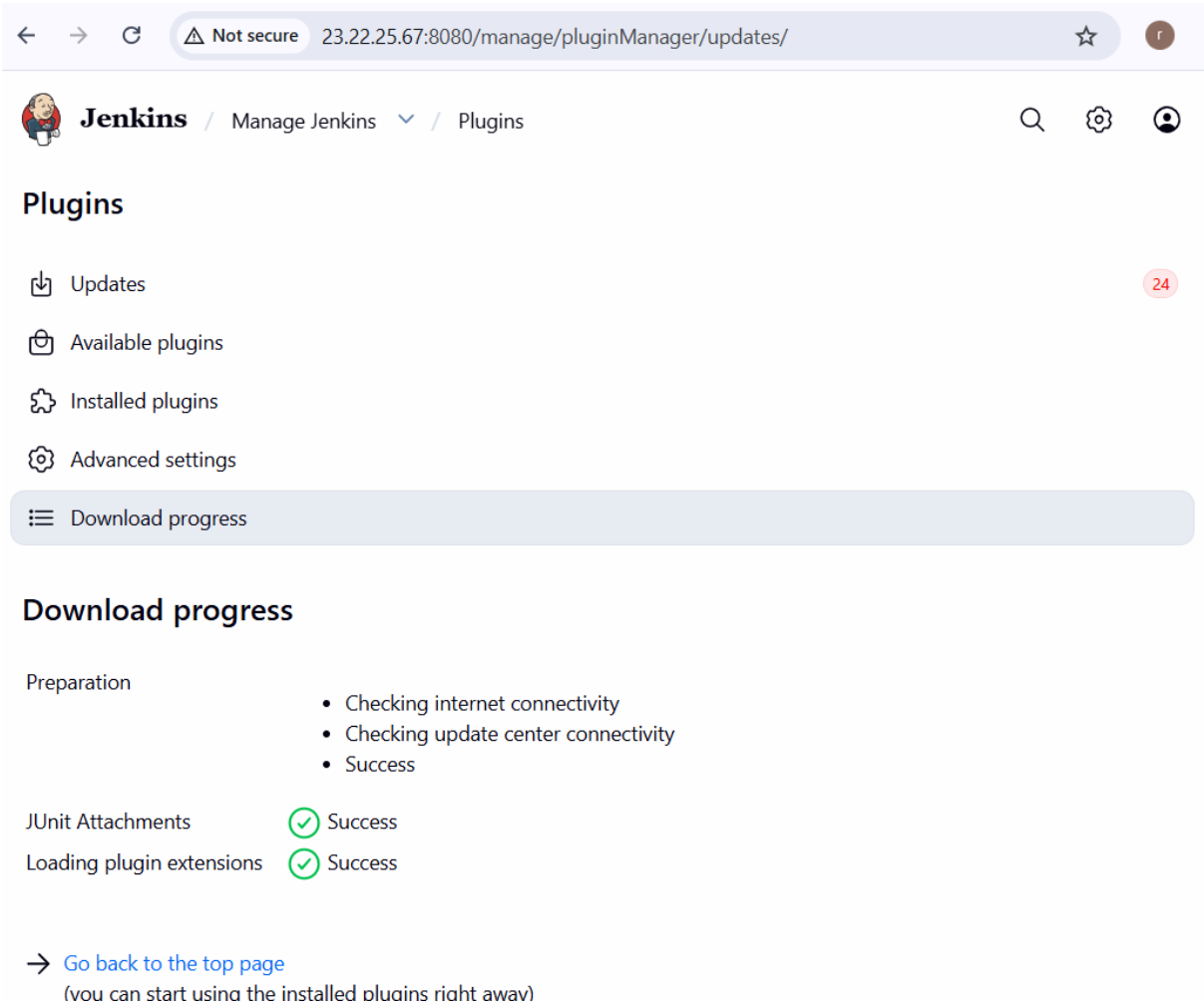
Service	**URL**
Jenkins	http://23.20.228.199:8080
SonarQube	http://44.203.82.139:9000
Nexus	http://13.218.184.207:8081

You will also need:

- Git
- A compatible JDK
- Maven, or the repository's Maven Wrapper
- Jenkins Pipeline and Git plugins

- SonarQube Scanner for Jenkins
- OWASP Dependency-Check
- A SonarQube token stored securely in Jenkins Credentials

Make sure the Jenkins build agent has enough disk space, memory and outbound network access. The service URLs above are from your earlier setup and have not been live-checked.



4. Step 2 — Verify Java, Maven and Git on the Jenkins agent

Run these commands on the machine that will execute the pipeline:

java -version

mvn -version

git --version

If Maven is missing on an Amazon Linux/RHEL-based machine, install it along with Git and a suitable JDK:

sudo dnf install -y git maven java-21-amazon-corretto-devel

Then verify the installed versions again. Use the Java version required by the repository's Maven configuration.

6. Step 4 — Create a Declarative Pipeline job

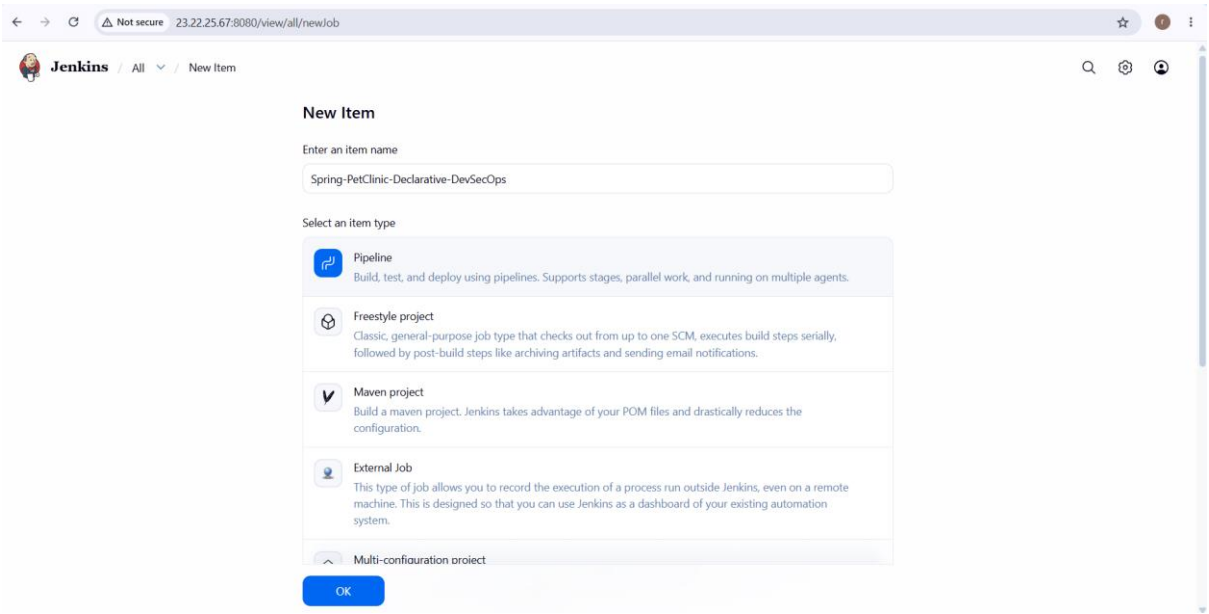
1. Create a job

In Jenkins, click New Item.

2. Enter the job name

Use Spring-PetClinic-Declarative-DevSecOps, select Pipeline, and click OK.

For the initial setup, we will keep the Jenkinsfile directly in the Jenkins job configuration. Later, you can store it in GitHub for version control. Jenkins supports both approaches.



5. Step 3 — Configure SonarQube in Jenkins

If you want SonarQube analysis, configure it before running the complete pipeline.

1. Log in to SonarQube at http://44.203.82.139:9000.
2. Generate a token from your account's security settings.
3. In Jenkins, open Manage Jenkins → Credentials.

4. Add a Secret text credential with ID sonarqube-token.
5. Open Manage Jenkins → System.
6. Under SonarQube servers, add a server configuration:
    - Name: SonarQube
    - Server URL: http://44.203.82.139:9000
    - Authentication token: select the credential you created.
7. Save the configuration.

For a Quality Gate stage, configure a SonarQube webhook pointing to:

http://23.20.228.199:8080/sonarqube-webhook/

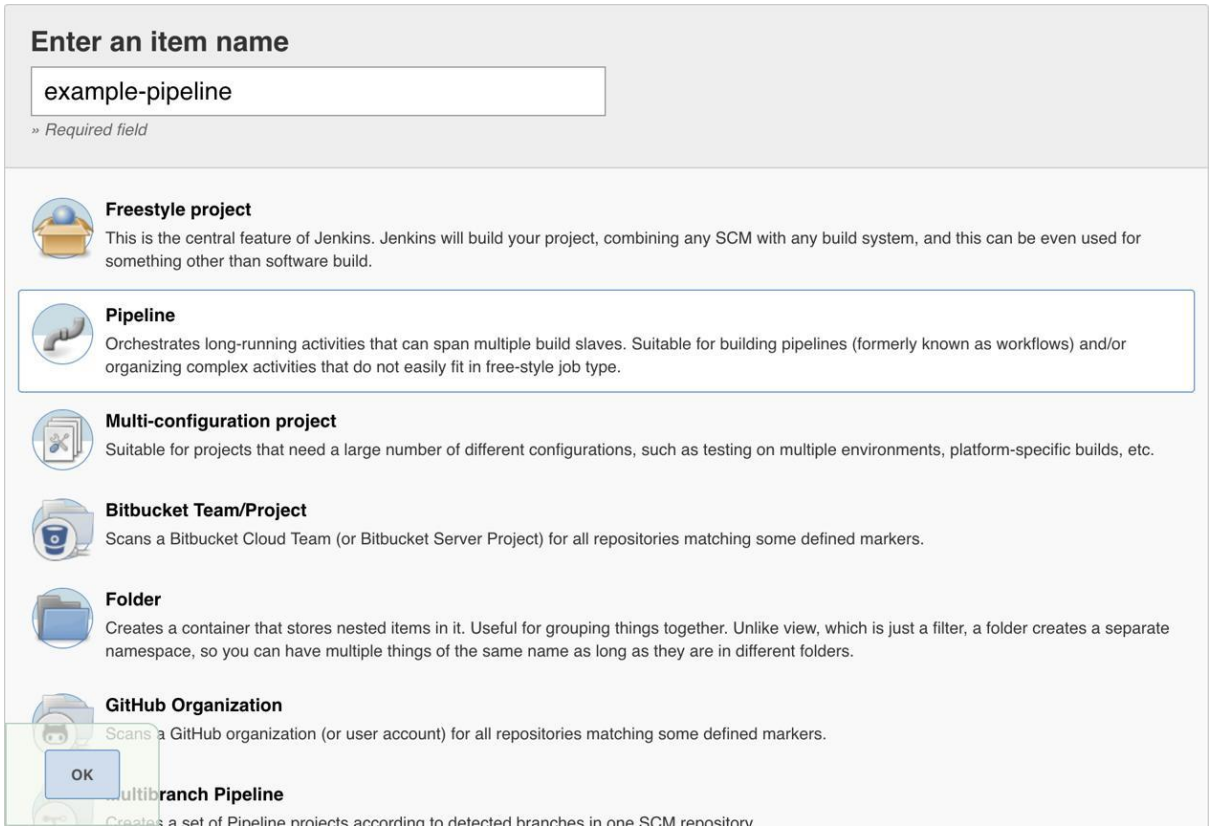
SonarQube requires this webhook for Jenkins to receive the Quality Gate result through waitForQualityGate. Ensure SonarQube can reach Jenkins at that URL.



Sonar Documentation

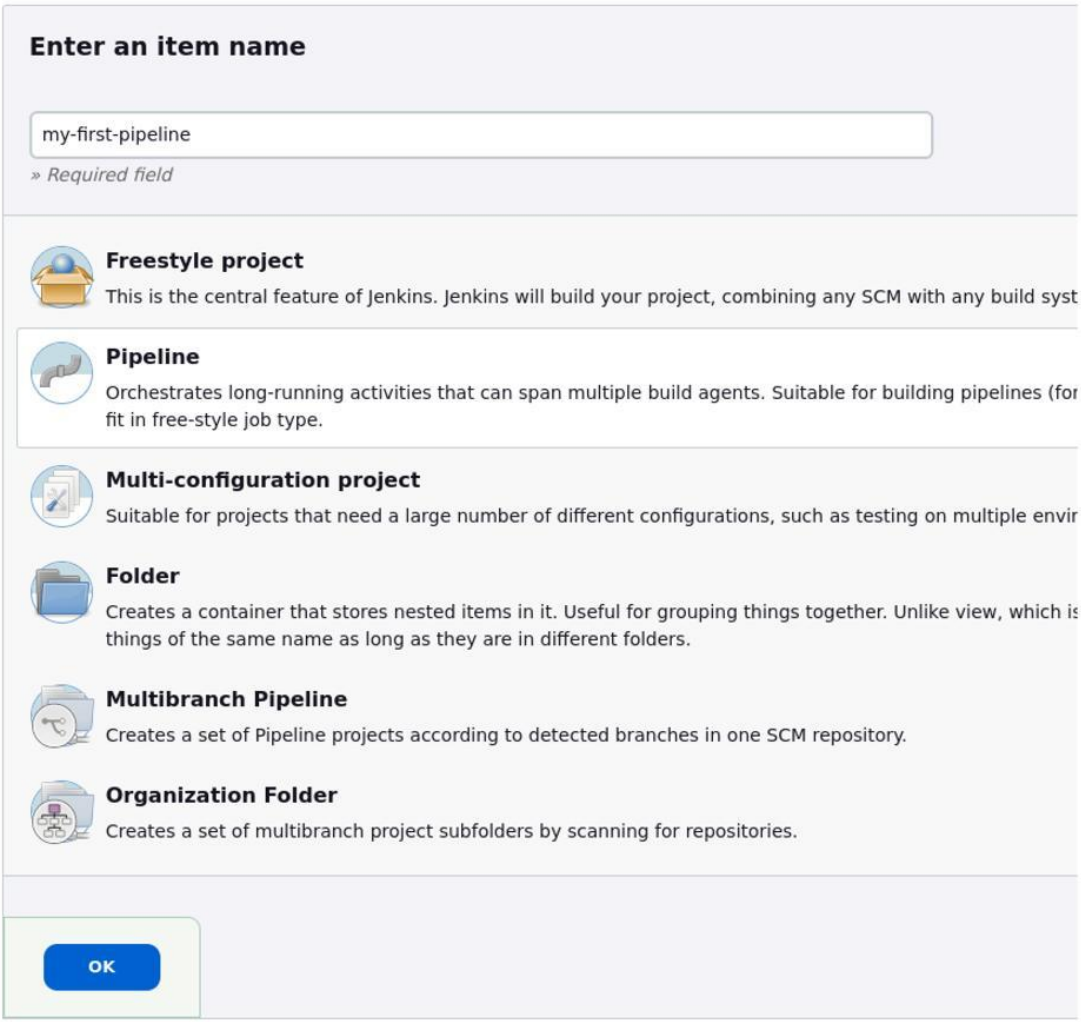
+1

6. Step 4 — Create a Declarative Pipeline job



1. Create a job

In Jenkins, click New Item.



LabEx

2. Enter the job name

Use Spring-PetClinic-Declarative-DevSecOps, select Pipeline, and click OK.

For the initial setup, we will keep the Jenkinsfile directly in the Jenkins job configuration. Later, you can store it in GitHub for version control. Jenkins supports both approaches.

Jenkins

7. Step 5 — Add the Declarative Pipeline code

Scroll to the Pipeline section.

- Definition: Pipeline script

- Script: paste the following code.

This first version performs Git checkout, build, testing, Dependency-Check, and artifact archiving. It intentionally leaves SonarQube as a separate next step so you can confirm the core build works first.

```
pipeline {
    agent any

    options {
        timestamps()
        timeout(time: 60, unit: 'MINUTES')
        skipStagesAfterUnstable()
    }

    environment {
        SONAR_PROJECT_KEY = 'spring-petclinic'
        SONAR_SERVER = 'SonarQube'
    }

    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/rajkumari-hub/spring-petclinic.git'

                sh '''
                    echo "===== Source Code Checkout ====="
```

```
            git log -1 --oneline
            ls -la
        '''
    }
}

stage('Build and Test') {
    steps {
        sh '''
            set -eu

            if [ -f ./mvnw ]; then
                chmod +x ./mvnw
                ./mvnw -B clean verify
            else
                mvn -B clean verify
            fi
        '''
    }

    post {
        always {
            junit testResults: '**/target/surefire-reports/*.xml',
                 allowEmptyResults: true
        }
    }
```

```
}

stage('OWASP Dependency Check') {
    steps {
        sh '''
            set -eu

            if [ -f ./mvnw ]; then
                ./mvnw -B org.owasp:dependency-check-maven:12.1.0:check \
                  -DfailBuildOnCVSS=8 \
                  -Dformats=HTML,XML
            else
                mvn -B org.owasp:dependency-check-maven:12.1.0:check \
                  -DfailBuildOnCVSS=8 \
                  -Dformats=HTML,XML
            fi
        '''
    }
}

stage('SonarQube Analysis') {
    steps {
        withSonarQubeEnv("${SONAR_SERVER}") {
            sh '''
                set -eu
```

```
            if [ -f ./mvnw ]; then
              ./mvnw -B \
                org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
                -Dsonar.projectKey="${SONAR_PROJECT_KEY}"
            else
              mvn -B \
                org.sonarsource.scanner.maven:sonar-maven-plugin:sonar \
                -Dsonar.projectKey="${SONAR_PROJECT_KEY}"
            fi
          '''
        }
      }
    }

    stage('SonarQube Quality Gate') {
      steps {
        timeout(time: 10, unit: 'MINUTES') {
          script {
            def qualityGate = waitForQualityGate()
            if (qualityGate.status != 'OK') {
              error "SonarQube Quality Gate failed: ${qualityGate.status}"
            }
          }
        }
      }
    }
```

```
    stage('Verify Build Artifact') {
        steps {
            sh '''
                set -eu
                echo "===== Generated JAR Files ====="
                find . -type f -path '*/target/*.jar' \
                 ! -name '*-sources.jar' \
                 ! -name '*-javadoc.jar' -print

                find . -type f -path '*/target/*.jar' \
                 ! -name '*-sources.jar' \
                 ! -name '*-javadoc.jar' | grep -q .
            '''
        }
    }
}

post {
    always {
        archiveArtifacts artifacts: '**/target/*.jar, **/target/dependency-check-report.*',
                         fingerprint: true,
                         allowEmptyArchive: true

        echo 'Pipeline execution completed. Check the stage results above.'
    }
```

```
        success {
            echo 'DevSecOps pipeline completed successfully.'
        }

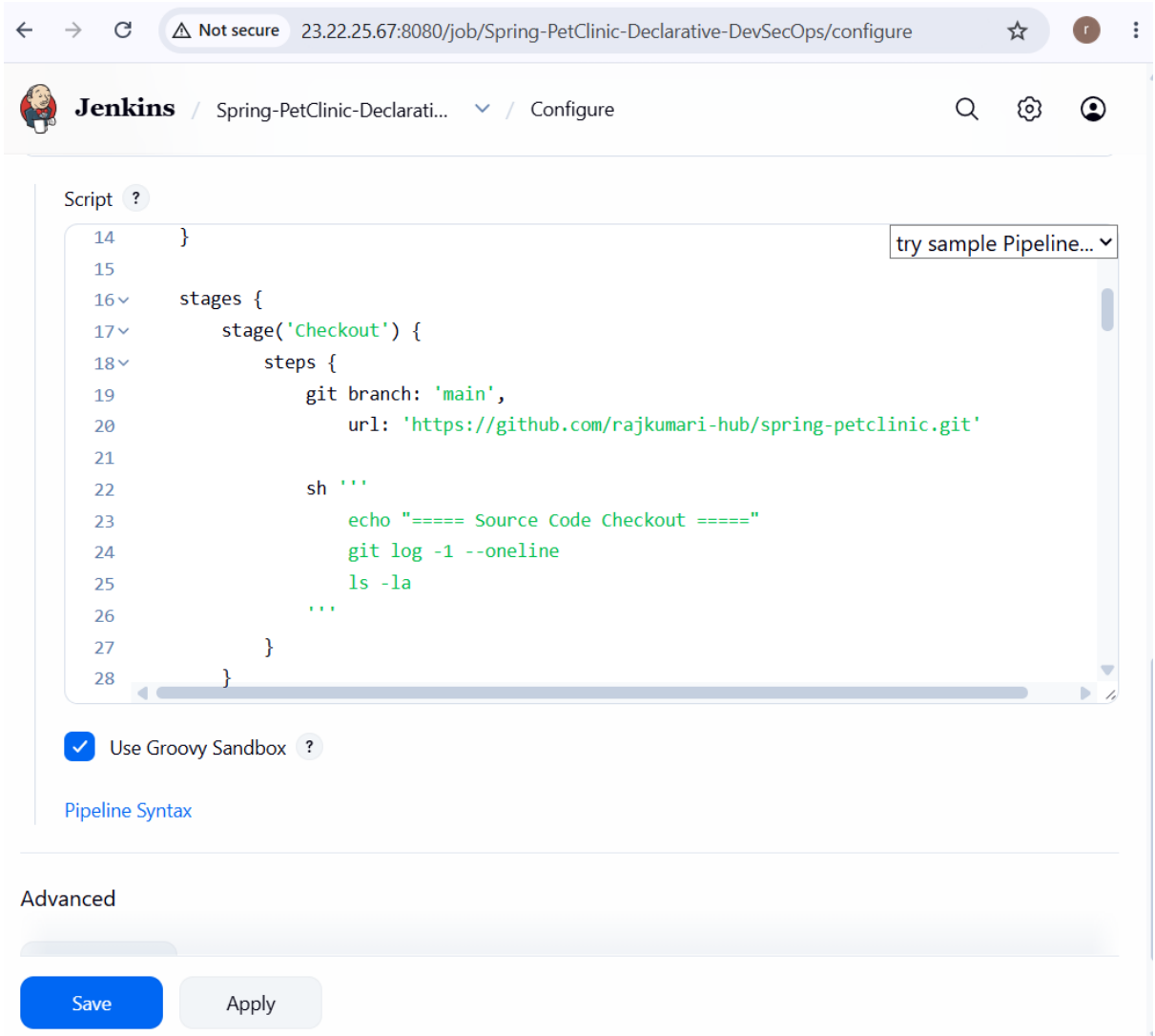
        failure {
            echo 'Pipeline failed. Review Console Output and the failed stage.'
        }
    }
}
```

Important configuration notes

- The code assumes the source branch is main. Change it if your repository uses another branch.
- The Dependency-Check stage scans for known dependency vulnerabilities and sets a failure threshold of CVSS 8. A finding at or above this threshold can fail the build. Its first run can take a long time while vulnerability data is downloaded.

dependency-check-maven

- The SonarQube analysis requires the global Jenkins server name to be exactly SonarQube.
- The Quality Gate stage requires the SonarQube webhook configured in Step 3 and the Jenkins SonarQube plugin.
- If your project has no test reports, Jenkins will warn that none were found, but the allowEmptyResults setting prevents that warning alone from failing the build.



8. Step 6 — Save and run the pipeline

1. Click Save.
2. Open the Spring-PetClinic-Declarative-DevSecOps job.
3. Click Build Now.
4. Click the build number in Build History.
5. Select Console Output to view logs.

Your stages should appear in this order:

Checkout

Stage 1

Build and Test

Stage 2

OWASP Dependency Check

Stage 3

SonarQube Analysis

Stage 4

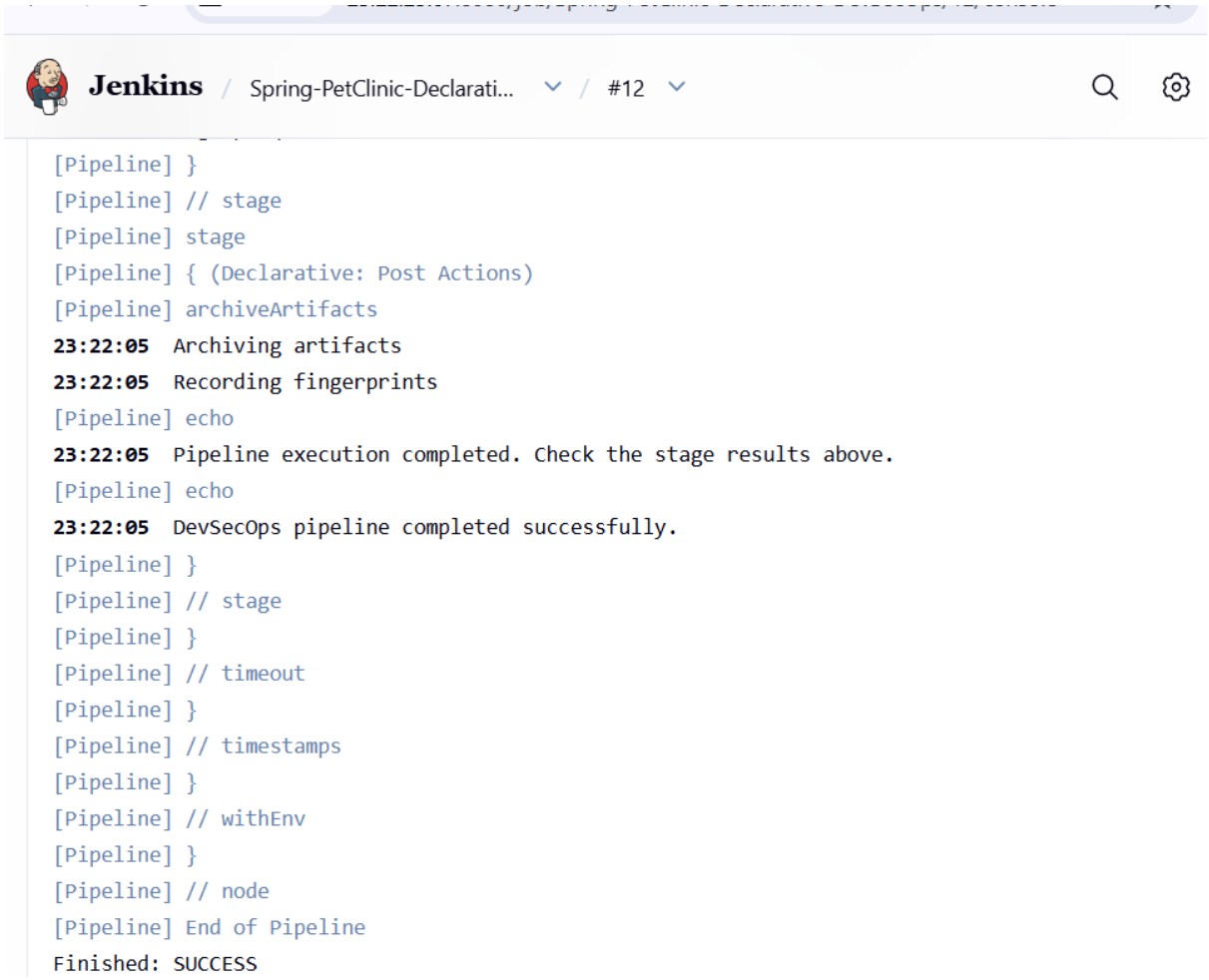
SonarQube Quality Gate

Stage 5

Verify Build Artifact

Stage 6

These are the expected stages, not a claim that your job has already passed.



9. Step 7 — Verify the results

A. Check the build

In Jenkins, open the build's Console Output and confirm that Maven reports BUILD SUCCESS.

B. Check the security scan

The Dependency-Check report should be generated under the Maven target directory. Open the build page and look under Build Artifacts for dependency-check-report.html or dependency-check-report.xml.

If the scanner fails while downloading its vulnerability database, check network access and NVD configuration before interpreting that failure as an application vulnerability.

C. Check SonarQube

Open http://44.203.82.139:9000, select the spring-petclinic project, and review its code-quality and security findings.

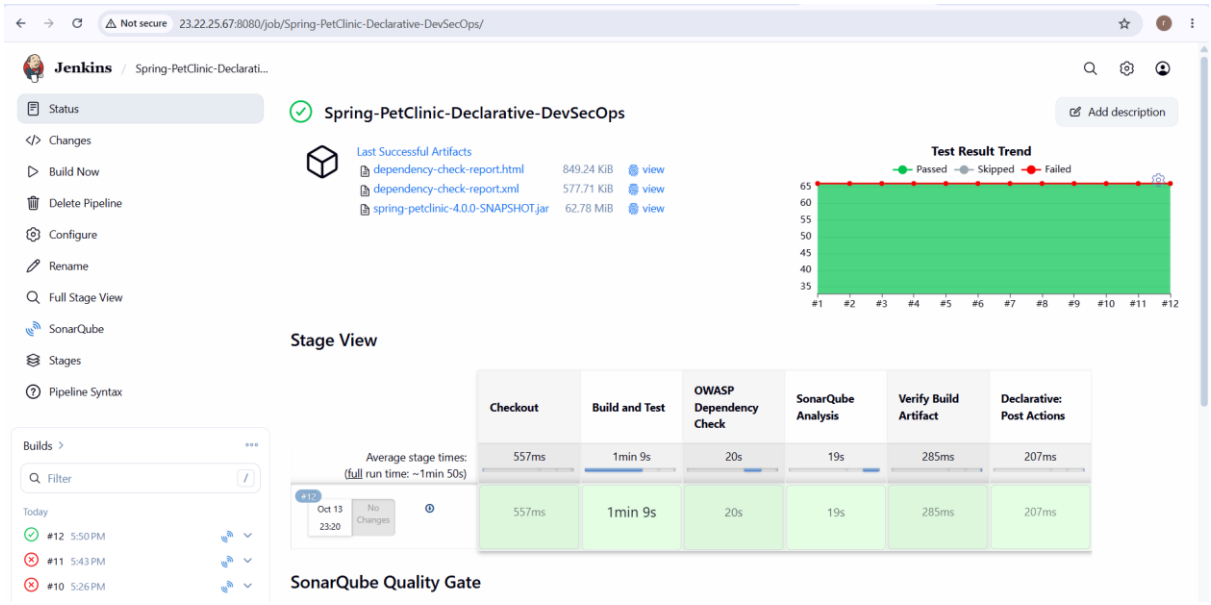
D. Check the JAR

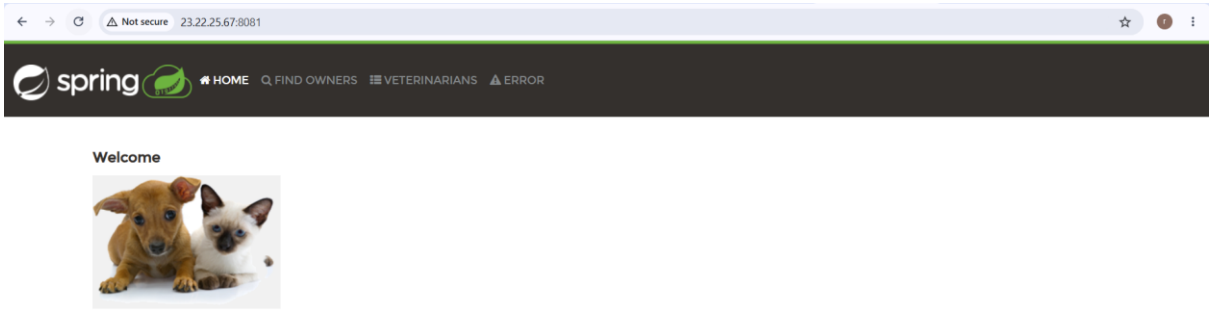
On the Jenkins build page, open Build Artifacts and check the archived JAR files.

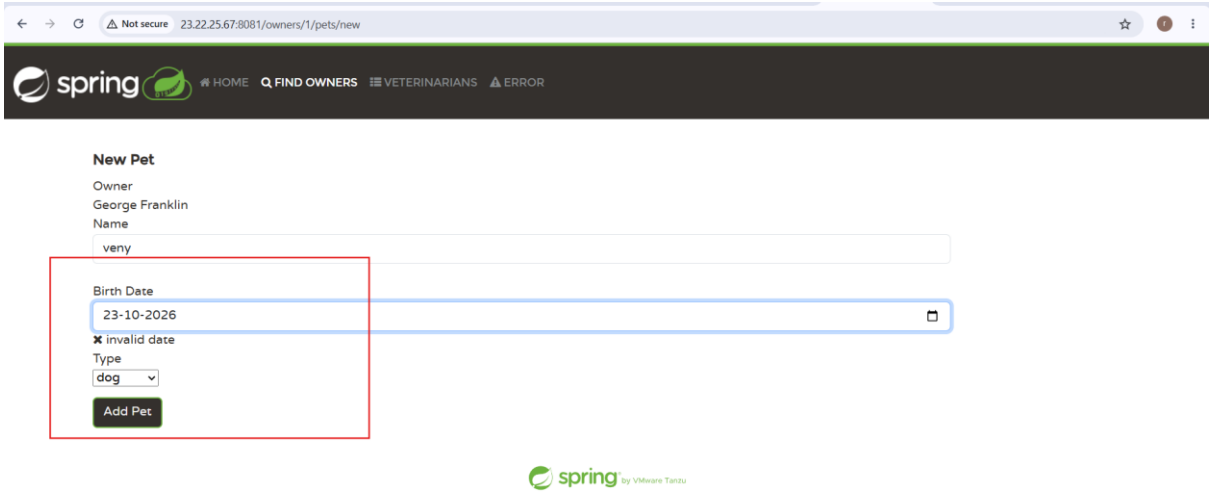
10. Common errors and solutions

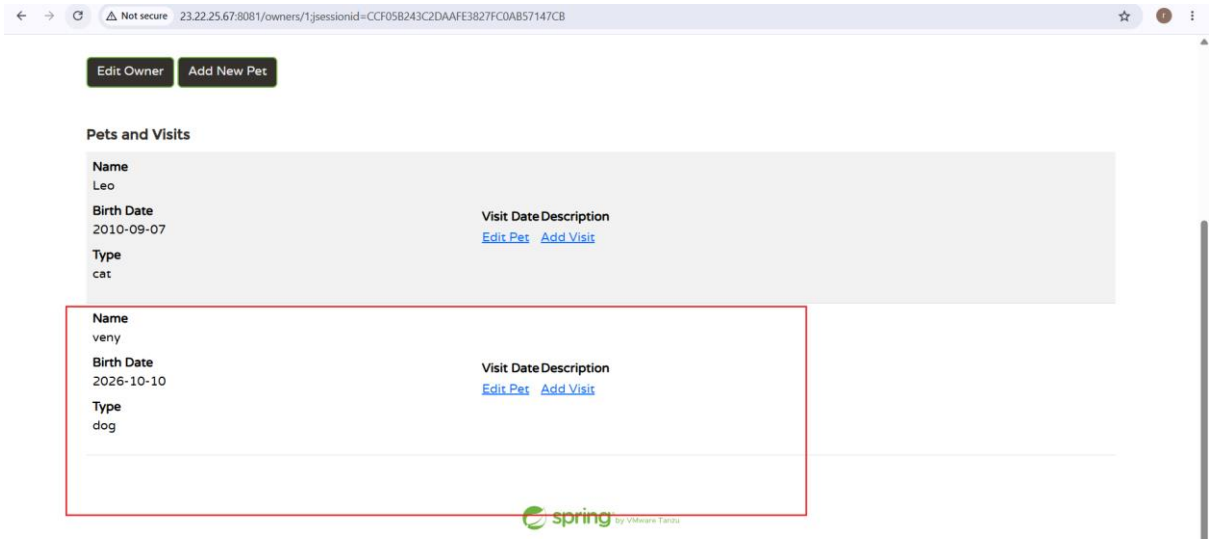
Error	Solution
mvn: command not found	Install Maven or use mvnw.
Git checkout fails	Verify repository URL, branch and Git plugin.
No such DSL method 'withSonarQubeEnv'	Install or update the SonarQube Scanner for Jenkins plugin.
No such DSL method 'waitForQualityGate'	Verify the SonarQube plugin installation and restart Jenkins if required.
Quality Gate times out	Check that the SonarQube webhook can reach Jenkins and inspect the analysis task.
Dependency-Check fails	Review NVD access, API-key configuration, network connectivity and scanner logs.

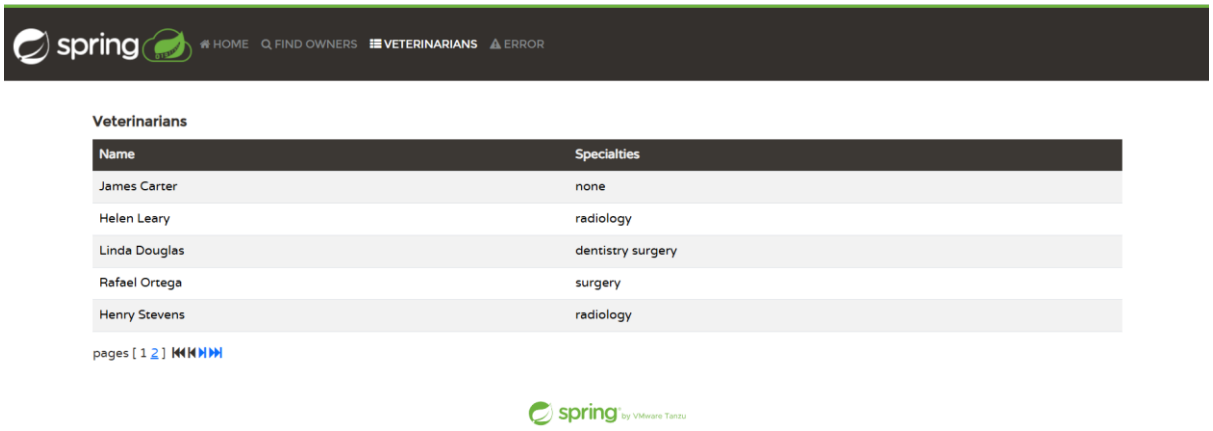
Build fails on CVSS threshold	Review the HTML report and address high-severity vulnerabilities; don't suppress genuine findings just to obtain a green build.
No JAR found	Check the Maven packaging configuration and build output.






Not secure 23.22.25.67:8081/owners/1/pets/new
spring
HOME
FIND OWNERS
VETERINARIANS
ERROR
New Pet
Owner
George Franklin
Name
veny
Birth Date
23-10-2026
invalid date
Type
dog
Add Pet
spring by VMware Tanzu


Not secure 23.22.25.67:8081/owners/1;jsessionid=CCF05B243C2DAAFE3827FC0AB57147CB
Edit Owner
Add New Pet
Pets and Visits
Name
Leo
Birth Date
2010-09-07
Type
cat
Visit Date
Description
Edit Pet
Add Visit
Name
veny
Birth Date
2026-10-10
Type
dog
Visit Date
Description
Edit Pet
Add Visit
spring by VMware Tanzu


spring
HOME
FIND OWNERS
VETERINARIANS
ERROR
Veterinarians
Name
Specialties
James Carter
none
Helen Leary
radiology
Linda Douglas
dentistry surgery
Rafael Ortega
surgery
Henry Stevens
radiology
pages [1 2]
spring by VMware Tanzu

spring HOME FIND OWNERS VETERINARIANS ERROR



Something happened...

An internal server error occurred.

spring by VMware Tanzu